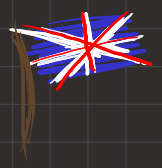
DISTRIBUITED SYSTEMS NOTES



Carlo Da Rome NOV. 5. 2026

CH. 1 G.P.E.

QUESTION 1: WHAT IS A DISTRIBUITED SYSTEM?

ANSWER: A DISTRIBUITED SYSTEM IS A SET OF COMPUTERS THAT COOPERATE TO EXECUTE OPERATIONS OR TO STORE DATAS, THEY DON'T USE CLOCK FOR SINCRONIZATION, THE SINCRONIZATION IS POSSIBLE THROUGH MESSAGES.

- A LOT OF PROBLEMS IN THIS TYPE OF SYSTEM CAN BE TRACED BACK TO G.P.E. (global predicate evaluation) THAT IS EVALUATE IF THE SYSTEM SATISFIES A CONDITION.
- AN ASINCRONOS DYSTRIBUITED SYSTEM (A.D.S.) IS A SYSTEM WITH THE FOLLOWING PROPERTIES:

1. THE M PROCESSES IN THE SYSTEM IS REPRESENTED WITH THE SET $(P_1, \dots, P_m)$.
2. THE COMMUNICATION CHANNELS IS BASICALLY NOT FIFO, THIS MEANS THAT THE ORDER OF SENDED MESSAGES COULD BE DIFFERENT TO THE RECIEVE ORDER.
3. PROCESSES ARE STRONG CONNECTED, THIS MEANS THAT PROCESSES ARE ALL CONNECTED EACH OTHER.
4. CHANNELS ARE RELIABLE... WE CAN ASSUME THAT WE CANNOT HAVE DATA LOST OR COMPROMIZED.

5. SINCRONIZATION PASSES ONLY THROUGH MESSAGES

THIS IS A THEORETICAL MODEL, IT DOEST EXIST IN REALITY 'CAUSE IS NOT POSSIBLE TO REALIZE THAT BUT IT IS A GOOD ABSTRACTION BECAUSE.

1. IS VERY SIMPLE AND DOESN'T HAVE ANY BOUND IN TERM OF SPEED OR RETARD, IF ONE SOLUTION IS GOOD ON THIS MODEL IT IS ALSO GOOD IN REAL LIFE.

2. SOME LAYERS IN REAL SYSTEMS FOLLOW THIS MODEL (RULE 4 IS IMPOSSIBLE TO SATISFIES IN REAL LIFE)

- IN A DISTRIBUITED COMPUTATION WE CAN HAVE 3 TYPE OF EVENTS

1. INTERNAL EVENT.

IT ONLY CHANGE THE STATE OF THE PROCESS THAT EXECUTES THE EVENT.

2. EXTERNAL EVENT.

2.1 SEND(m)

2.2 RECIEVE(m)

THESE TWO TYPE OF EVENTS ATTACH A MESSAGE INDICATED WITH THE LECTER "m"

CH.2 GLOBAL STATES, CUT AND CONSISTENCY

QUESTION 2: WHAT IS CONSISTENCY?

ANSWER: IN A D.S. WE HAVE CONSISTENCY WHEN THE MONITOR PROCESS p_0 (PROCESS THAT MAKE G.P.C.) RECIEVE COMPATIBLE STATES.

- HISTORY.

IN DISTRIBUITED SYSTEM WE HAVE TWO TYPE OF

HISTORY...

1. LOCAL HISTORY.

IS THE SET OF THE EVENTS EXECUTED BY A SPECIFIC PROCESS P_i

$h_i = e_i^1 e_i^2 e_i^3 \dots$ POTENCIALLY INFINITE...

↑ WHERE i IS REFERRED TO THE PROCESS P_i

2. GLOBAL HISTORY.

IS THE SET OF ALL THE EVENTS EXECUTED BY ALL THE PROCESSES IN THE DISTRIBUTED SYSTEM.

$H = h_1 \cup h_2 \cup h_3 \cup \dots \cup h_m$

$H = e_1^1 \dots e_1^{\dots} \cup e_2^1 \dots e_2^{\dots} \cup [\dots]$

- CAUSAL PRECEDENCE.

THE CAUSAL PRECEDENCE "$\rightarrow$" IS A RELATION ON THE EVENTS IN H

IF $e \rightarrow e' \Rightarrow e$ INFLUENCE e' AND FOLLOWS THE FOLLOWING PROPERTIES:

1. IF e, e' IS EVENT OF THE SAME PROCESS P_i AND e PRECEDES e' IN THE LOCAL HISTORY h_i THEN $e \rightarrow e'$.
2. IF e = SEND(m) AND e' = RECIEVE(m) THEN $e \rightarrow e'$
3. IF $e \rightarrow e'$ AND $e' \rightarrow e''$ THEN $e \rightarrow e''$.

- GLOBAL STATES, CUT, RUN

- TO PERFORM A G.P.E. THE MONITOR p_0 REQUIRES THE GLOBAL STATE OF THE SISTEM... THE GLOBAL STATE Σ IS DEFINED AS $\Sigma = \sigma_1, \sigma_2, \dots, \sigma_m$ WHERE σ_i IS THE LOCAL STATE OF P_i.
- A CUT IS A SUBSET IN THE GLOBAL HISTORY.

$C = h_1^{e_1} \cup \dots h_m^{e_m}$.

THE TUPLE $(e_1,\dots,e_m)$ INDICATE THE LAST EVENT TO CONSIDER IN THE LOCAL HISTORY.

- A RUN IS A TOTAL ORDER ON H, IF e PRECEDES e' IN LOCAL HISTORY h_i OF p_i THEN e PRECEDES e' IN R.
- A CUT IS CONSISTENT IF LEFT CLOSED OR RATHER: IF $\forall e, e' \in H$ AND $e' \in C$ AND $e \to e' \Rightarrow e \in C$.
- A RUN IS CONSISTENT IF $\forall e, e'$ $e \to e' \Rightarrow$ e APPEARS BEFORE e' IN R.

CH. 3 MONITORING A D.S.

TO MONITOR A D.S. WE CAN APPLY TWO TYPE OF PREDICATE... ACTIVE MONITOR OR PASSIVE MONITOR.

ACTIVE MONITOR.

IN SYSTEMS WITH ACTIVE MONITOR, THE MONITOR PROCESS p_0 ASKS CONTINOUSLY TO THE OTHER PROCESSES THEIR LOCAL STATE. WE CAN NOTICE SOME PROBLEMS IN THIS TYPE OF SYSTEM.

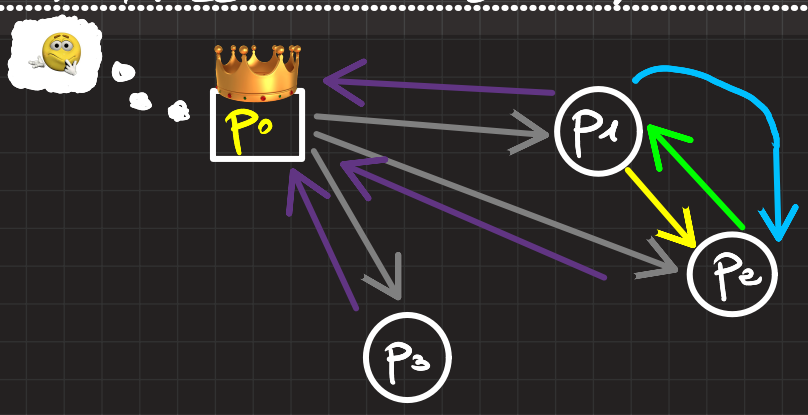
1. GLOBAL STATES THAT THE MONITOR RETRIEVE CAN BE INCONCOSISTENT AND ENCREASING THE FREQUENCY OF THE REQUESTS THE PROBLEM IS NOT FIXED.

2. GHOST DEADLOCK

WE ALSO COULD HAVE GHOST DEADLOCK...

BASIC EXAMPLE OF GHOST DEADLOCK.

EXAMPLE 1.



IN THE EXAMPLE WHE HAVE THE FOLLOWING SITUATION:

P_1 SEND TO P_2 A MESSAGE AND IT WAIT FOR A CONFIRMATION MESSAGE FROM P_2

P_2 SEND TO P_1 THE CONFIRMATION MESSAGE ... AND WAIT FOR A CONFIRMATION MESSAGE FROM P_1 ...

IF THE MONITOR P_0 SENDS THE STATE REQUEST IT CAN ARRIVES TO P_1 AND P_2 IN THE FOLLOWING MOMENT...

P_1 : WAITING FOR P_2 CONFIRMATION MESSAGE
P_2 : WAITING FOR P_1 CONFIRMATION MESSAGE

(THIS IS CAUSED BY THE IMPREVEDIBLE RETARDS ON THE COMMUNICATION LINE)

THE GLOBAL STATE RETRIVED IS AN INCONSISTENT STATE AND IT CONTAINS A GHOST DEADLOCK, OR RATHER A DEADLOCK THAT DOESN'T EXIST IN THE SYSTEM BUT EXIST IN G.P.E..

- PASSIVE MONITOR.

IN SYSTEM WITH PASSIVE MONITOR THE MONITOR PROCESS P_0 DOESN'T ASK TO THE OTHER THEIR STATE BUT THE PROCESSES IN D.S. SENDS TO THE MONITOR ONLY RELEVANT INFORMATIONS FOR THE G.P.E.

TO AVOID UNCONSISTENT CUT WE INTRODUCE FIFO RULE

- FIFO RULE

EVERY PROCESS IN THE DISTRIBUITED SYSTEM HAS 2 LEVEL 1) DELIVERY LEVEL 2) APPLICATION

LEVEL.
ALL THE MESSAGES ARRIVE IN THE DELIVERY LEVEL AND DELIVERED TO THE APPLICATION LEVEL FOLLOWING THE ORDER DESCRIBED BY THE **FIFO RULE** OR RATHER:

A COMMUNICATION BETWEEN p_i AND p_j FOLLOW THE **FIFO DELIVERY RULE** IF $\forall m, m'$

$$SEND_i(m) \rightarrow SEND_i(m') \Rightarrow DELIVER_j(m) \rightarrow DELIVER_j(m')$$

where the pedice indicates the process that execute the event.

CH. 4 CLOCK

- UNTIL NOW WE HAVE ANALIZED DISTRIBUITED SYSTEM WITH NO CLOCK, NOW, SUPPOSE TO HAVE A SYSTEM WHERE THE MONITOR RECIEVES ALL THE MESSAGES EXCHANGED BETWEEN THE PROCESSES WITH AN ATTACHED **TIMESTAMP** AND P_0 STORE THESE MESSAGES IN A QUEUE AND DELIVERS THESE PERIODICALLY TO THE APPLICATION LEVEL.
THIS TYPE OF SYSTEMS SATISFIES **CLOCK CONDITION**

- **CLOCK CONDITION**
A FUNCTION T THAT ASSOCIATES A TIMESTAMP TO AN EVENT SATISFIES THE CLOCK RULE IF $e \rightarrow e' \Rightarrow T(e) < T(e')$.
BUT THE CLOCK RULE CAN BE SATISFIED BY OTHER TYPE OF SYSTEMS...

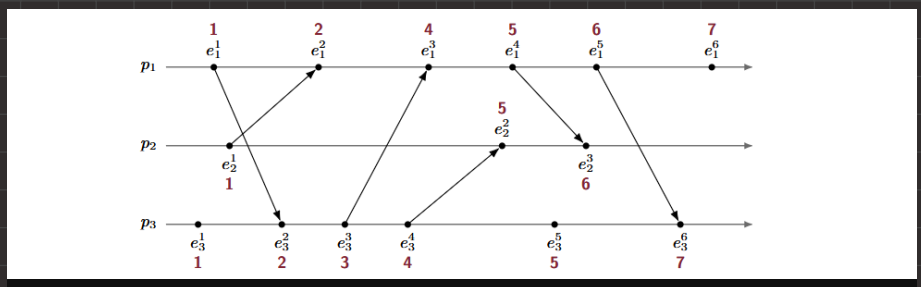
- LAND PORT LOGICAL CLOCKS.
SUPPOSE TO HAVE A COUNTER "LC" FOR EACH PROCESS p_i IN A DISTRIBUITED SYSTEM.
WHEN AN EVENT OCCOURRES THE LC ASSOCIATED TO THE EVENT FOLLOW THE FOLLOWING RULE:

$$LC(e_i) = \begin{cases} LC \mathrel{+}= 1 & \leftarrow \text{INTERNAL / SEND EVENT} \\ \max\{LC, TS(m)\} + 1 & \leftarrow \text{RECEIVE EVENT} \end{cases}$$

THIS TYPE OF SYSTEM ALSO SATISFIES THE CLOCK CONDITION...

TO BETTER COMPREND FOLLOW THE FOLLOWING EXAMPLE...

EXAMPLE 3.



- The first events e_1^1, e_1^2, and e_1^3 are either send events or internal events, so their clock values are 1. The requests sent in e_1^1 and e_1^2 carry a timestamp of 1.
- Event e_2^1 receives the request from process p_2 with timestamp 1: $\max\{1, 1\} + 1 = 2$. Similarly, e_3^3, which receives the request from p_1, gets a value of 2.
- Process p_3 sends a response, $LC(e_3^3) = 3$, and then sends a request to p_2, $LC(e_4^3) = 4$.
- Event e_3^1 receives the response from p_3 with timestamp 3: $\max\{2, 3\} + 1 = 4$. The clock of process p_1 jumps from 2 to 4 to catch up with p_3's past.
- Event e_2^2 receives the request from p_3 with timestamp 4: $\max\{1, 4\} + 1 = 5$.
- Process p_1 proceeds with the response to p_2, $LC(e_4^1) = 5$, the request to p_3, $LC(e_5^1) = 6$, and the internal event e_6^1, which has a value of 7.
- Event e_3^2 receives the response from p_1 with timestamp 5: $\max\{5, 5\} + 1 = 6$. Event e_5^3 is internal and has a value of 5; event e_6^3 receives the request sent in e_5^1 with timestamp 6: $\max\{5, 6\} + 1 = 7$. The clock of process p_3 jumps from 5 to 7 and never assumes the value 6.

- THESE TWO TYPE OF SYSTEM DON'T HAVE GAP DETECTION BECAUSE THEY CANNOT RECOGNIZE IF AN EVENT WITH LOWER TIMESTAMP THAN THE HIGHEST TIMESTAMP THAT THEY HAVE IN THEIR HAND CAN ARRIVE...

QUESTION 3: WHAT IS GAP DETECTION?

ANSWER: A SYSTEM HAS GAP DETECTION IF GIVEN $LC(e) < LC(e')$ CAN RECOGNIZE IF EXIST e'' WITH $LC(e) < LC(e'') < LC(e')$.

- HOW TO ADD GAP-DETECTION TO OUR SYSTEM?

WE CAN IMPLEMENT THIS IN A VERY SIMPLE WAY.

1) ADD FIFO CHANNELS IN EACH PROCESS OF THE D.D.

2) p_0 DELIVERS A MESSAGE ONLY IF IS STABLE.

- STABLE MESSAGE.

A MESSAGE m WITH TIMESTAMP $TS(m)$ IS STABLE FOR p_0 IF IT RECIEVE AT LEAST A MESSAGE m_1 WITH TIME STAMP $TS(m_1) > TS(m)$ FROM EACH PROCESS IN THE DISTRIBUITED SYSTEM.

WHY IT WORKS ???

BECAUSE IF p_0 RECIEVE A MESSAGE m WITH TIMESTAMP 4 FROM p_1, A MESSAGE m_1 WITH TIMESTAMP 5 FROM p_2 AND A MESSAGE m_{11} WITH TIMESTAMP 6 FROM p_3 IN A D.D. $\{p_0, p_1, p_2, p_3\}$, p_0 IS SURE TO NOT RECIEVE A MESSAGE WITH TIMESTAMP LOWER THAN 4 BECAUSE EACH PROCESS HAS A FIFO CHANNEL.

PROBLEM: A PROCESS CAN STOP SEND MESSAGES

SOLUTION: p_0 ASK TO STOPPED PROCESSES FOR CONFIRM MESSAGES, THESE RESPONSES WILL HAVE BIGGER TS THAN THE MESSAGES IN QUEUE.

Exercises 1

Alessandro Mei • 1 ott

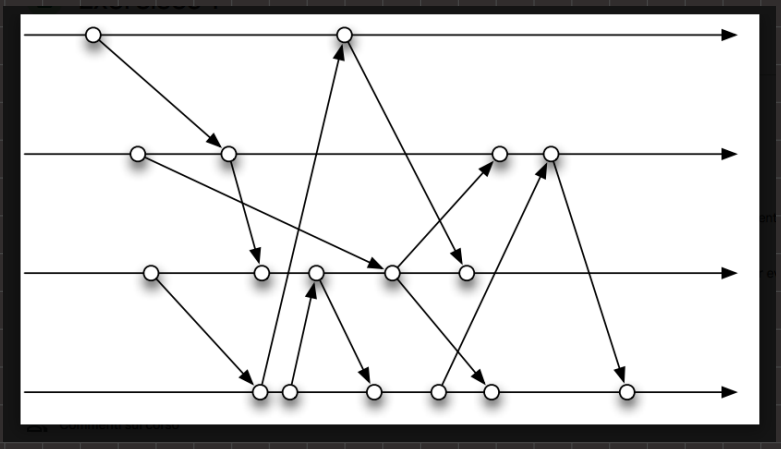
Exercise 1: Let C1 and C2 be two consistent cuts. Show that the intersection of C1 and C2 is a consistent cut.

Exercise 2: Let C1 and C2 be two consistent cuts. Show that the union of C1 and C2 is a consistent cut.

Exercise 3: Show that every consistent global state can be reached by some consistent run.

Exercise 4: Let C1 and C2 be two consistent cuts. If C1 is a subset of C2, then C2 is reachable from C1. (There exists a consistent run that reaches C1 and then reaches C2.)

Exercise 5: Label with proper logical clock all the events of the distributed computation in image vector.pdf. (You can consider events that receive a message and immediately send it as single events.)



EXERCISE 1

LET c_1 AND c_2 CONSISTENT CUTS SHOW THAT THE INTERSECTION $(c_1 \cap c_2)$ IS A CONSISTEN CUT...

- A CUT IS CONSISTENT IF IS LEFT CLOSED, OR RATHER, IF $\forall e, e' \in H, e \in C \wedge e \to e' \Rightarrow e \in C$ WHERE "H" IS GLOBAL HISTORY AND "C" IS A CUT.

WE HAVE TO DEMONSTRATE THAT:

$\forall e, e' \in H, e' \in (c_1 \cap c_2), e \to e' \Rightarrow e \in (c_1 \cap c_2)$... AND THIS IS TRUE BECAUSE IF $e' \in (c_1 \cap c_2) \Rightarrow (e' \in c_1) \wedge (e' \in c_2)$

NOW, IF $e \to e'$ AND $e' \in (c_1 \cap c_2) \Rightarrow (e' \in c_1) \wedge (e' \in c_2) \Rightarrow$

$e \in c_1 \wedge e \in c_2 \Rightarrow e \in (c_1 \cap c_2)$ WHERE $e, e' \in H$.

EXERCISE 2

LET c_1 AND c_2 CONSISTENT CUTS SHOW THAT THE UNION $(c_1 \cup c_2)$ IS A CONSISTEN CUT...

- A CUT IS CONSISTENT IF...

WE HAVE TO DEMONSTRATE THAT:

$\forall e, e' \in H, e' \in (c_1 \cup c_2), e \to e' \Rightarrow e \in (c_1 \cup c_2)$... AND THIS IS TRUE BECAUSE IF $e' \in (c_1 \cup c_2) \Rightarrow (e' \in c_1) \vee (e' \in c_2)$

NOW, IF $e \to e'$ AND $e' \in (c_1 \cup c_2) \Rightarrow (e' \in c_1) \vee (e' \in c_2) \Rightarrow$

$e \in c_1 \vee e \in c_2 \Rightarrow e \in (c_1 \cup c_2)$ WHERE $e, e' \in H$.